

Smart Student Assignment & Deadline Manager

StudyBoard — Complete Project Documentation & Prompt Book
Live: <https://mystudyboard.lovable.app>

Field	Detail
Project name	Smart Student Assignment & Deadline Manager (StudyBoard)
Type	Full-stack web application (SSR React)
Frontend	React 19 + TanStack Start v1 + TanStack Router + Vite 7
Styling	Tailwind CSS v4 + shadcn/ui, OKLCH design tokens
Data layer	TanStack Query v5
Backend	Lovable Cloud (Postgres + Auth + Row Level Security)
Auth	Email + password, role based (admin / student)
Roles	admin, student (separate user_roles table)
Published URL	https://mystudyboard.lovable.app
Document date	06 September 2026

1. Project Overview

StudyBoard ek academic productivity web app hai jo college students ke assignments, unke deadlines aur department notices ko ek hi jagah manage karta hai. App do tarah ke users support karta hai — Student aur Admin. Student apne assignments add/edit/delete karta hai aur notices padhta hai; Admin notices publish karta hai aur saare students ka overview dekhta hai.

Core problem it solves

- Students ke assignments alag-alag jagah (WhatsApp, notebook, LMS) bikhre rehte hain.
- Deadline kitni paas hai iska visual signal nahi milta — kaam last moment par jaata hai.
- Department notices ka koi single, reliable board nahi hota.

Solution highlights

- Ek dashboard jisme total / pending / completed / due-this-week / overdue statistics.
- Automatic Deadline Risk Engine jo har assignment ko DONE, OVERDUE, HIGH, MEDIUM ya LOW label deta hai.
- Search + status filter ke saath complete assignment CRUD.
- Admin-only notice board jo sabhi signed-in users ko turant dikhta hai.
- Role-aware navigation: Admin panel link sirf admin ko dikhta hai.

2. Feature List (module wise)

#	Module	Features
1	Landing page (/)	Hero section, feature cards, risk-engine explanation, CTA buttons for Login / Register, SEO meta tags
2	Auth (/auth)	Tabs for Login and Register, name+email+password signup, error handling (weak/leaked password, wrong credentials), auto redirect to dashboard on success
3	Dashboard (/dashboard)	5 stat cards, upcoming deadlines list with risk badges, latest notices preview, empty states
4	Assignments (/assignments)	Create, edit, delete, mark complete/pending, subject+title+description+deadline+priority fields, live search, status filter tabs, risk badge per card
5	Notices (/notices)	Read-only chronological notice board for students with author + timestamp
6	Admin panel (/admin)	Publish notice (title + message), delete notice, student roster with role tags; blocked with 'Access denied' screen for non-admins
7	App shell	Fixed dark sidebar, mobile hamburger drawer, sticky page header, profile card with role, sign out
8	Security	RLS on every table, security-definer has_role() function, no role stored on profile

3. Deadline Risk Engine (core logic)

Yeh app ka sabse important business rule hai. Har assignment ke status aur deadline se ek risk label calculate hota hai (file: `src/lib/assignments.ts`).

Condition	Risk label	Colour
status = Completed	DONE	Green (success)
Pending and deadline < today	OVERDUE	Red (destructive)
Pending and days left <= 1	HIGH	Red (destructive)
Pending and days left <= 3	MEDIUM	Amber (warning)
Pending and days left > 3	LOW	Blue (info)

```
export function deadlineRisk(a) {
  if (a.status === "Completed") return "DONE";
  const diff = daysUntil(a.deadline);
  if (diff < 0) return "OVERDUE";
  if (diff <= 1) return "HIGH";
  if (diff <= 3) return "MEDIUM";
  return "LOW";
}
```

daysUntil() local timezone me whole-day difference nikalta hai, isliye timezone shift se galat label nahi banta. Dashboard statistics bhi isi helper se bante hain: dueThisWeek = pending assignments jinke days left 0 se 7 ke beech hain, overdue = pending with days left < 0.

4. Technology Stack (detail)

Layer	Choice	Why
Framework	TanStack Start v1 (React 19, SSR)	File-based routing, server functions, fast Vite 7 build
Routing	TanStack Router (src/routes/*)	Type-safe links, per-route head() for SEO
Styling	Tailwind CSS v4 via src/styles.css	Token-first design system, no config file needed
Components	shadcn/ui (button, input, card, tabs, dialog, badge, select, sonner)	Accessible, fully themeable primitives
State/data	TanStack Query v5	Caching, invalidation after mutations, loading skeletons
Backend	Lovable Cloud (Postgres, Auth, RLS)	Managed database + auth without separate server
Icons	lucide-react	Consistent line icons
Fonts	Sora (headings) + Manrope (body)	Modern academic-professional pairing

5. Design System (UI/UX decisions)

Colour palette (OKLCH tokens in src/styles.css)

Token	Light value	Use
--primary	oklch(0.42 0.11 235)	Deep navy-blue — buttons, links, focus ring
--accent	oklch(0.78 0.14 72)	Warm amber — highlights, sidebar logo
--success	oklch(0.58 0.13 155)	DONE badge, completed state
--warning	oklch(0.75 0.15 70)	MEDIUM risk badge
--info	oklch(0.6 0.1 235)	LOW risk badge
--destructive	oklch(0.56 0.2 25)	OVERDUE / HIGH risk, delete actions
--sidebar	oklch(0.24 0.04 240)	Dark navy sidebar surface
--background	oklch(0.985 0.005 95)	Warm off-white page canvas

Typography

- Display font: Sora (500/600/700) — h1 to h4, letter-spacing -0.02em.
- Body font: Manrope (400–700) — paragraphs, labels, table text.
- Google Fonts <link> root route ke head() me load hoti hai (Tailwind v4 me remote @import allowed nahi).

Layout & interaction rules

- Radius scale 0.75rem base; cards surface-card utility use karte hain (border + soft shadow).
- Two custom shadows: --shadow-card (rest) aur --shadow-lift (hover/elevated).
- Hero gradient token: linear-gradient navy → blue → teal.

- Desktop par fixed 16rem sidebar, mobile par slide-in drawer + dark overlay.
- Sticky translucent header with backdrop blur; page title + subtitle + contextual action button.
- Dark mode tokens fully defined (.dark block) — har colour ka dark counterpart hai.
- Empty states, skeleton loaders aur toast feedback (sonner) har mutation par.

6. Database Schema

Table	Columns	Notes
profiles	id (uuid, PK → auth.users), name, email, created_at	Auto-created by trigger on signup
user_roles	id, user_id (→ auth.users), role (enum app_role), unique(user_id, role)	Roles NEVER stored on profiles — prevents privilege escalation
assignments	id, user_id, subject, title, description, deadline (date), priority, status, created_at	Priority: Low/Medium/High; Status: Pending/Completed
notices	id, title, message, created_by, created_at	Admin authored, readable by all signed-in users

Enum

```
CREATE TYPE public.app_role AS ENUM ('admin', 'student');
```

Auto-provision trigger

Naya user signup karte hi trigger on_auth_user_created chalta hai jo profile row banata hai aur default role 'student' assign karta hai.

```
CREATE FUNCTION public.handle_new_user() RETURNS TRIGGER
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public AS $$
BEGIN
  INSERT INTO public.profiles (id, name, email)
  VALUES (NEW.id,
    COALESCE(NEW.raw_user_meta_data->>'name', split_part(NEW.email, '@', 1)),
    COALESCE(NEW.email, ''))
  ON CONFLICT (id) DO NOTHING;
  INSERT INTO public.user_roles (user_id, role) VALUES (NEW.id, 'student')
  ON CONFLICT (user_id, role) DO NOTHING;
  RETURN NEW;
END; $$;
```

7. Security Model (RLS policies)

Har table par Row Level Security enabled hai aur har table ke liye explicit GRANT diye gaye hain. Role check ke liye SECURITY DEFINER function has_role() use hota hai taaki RLS recursion na ho.

```
CREATE FUNCTION public.has_role(_user_id uuid, _role public.app_role)
RETURNS boolean LANGUAGE sql STABLE SECURITY DEFINER
SET search_path = public AS $$
  SELECT EXISTS (SELECT 1 FROM public.user_roles
                WHERE user_id = _user_id AND role = _role)
$$;
```

Table	Policy	Rule
profiles	own profile read	id = auth.uid() OR has_role(auth.uid(),'admin')
profiles	own profile update	id = auth.uid() (both USING and WITH CHECK)
user_roles	own roles read	user_id = auth.uid() OR admin
assignments	students manage own	ALL commands where user_id = auth.uid()
assignments	admins view all	SELECT if has_role(auth.uid(),'admin')
notices	signed-in read	SELECT true for authenticated
notices	admins create	INSERT if admin AND created_by = auth.uid()
notices	admins delete	DELETE if admin

Important fix applied during build

Shuru me has_role() par EXECUTE permission revoke ho gaya tha, jisse RLS check fail hote the aur app me 'permission denied for function has_role' aata tha — role Student dikhta tha aur assignments load nahi hote the. Fix: GRANT EXECUTE ON FUNCTION public.has_role(uuid, public.app_role) TO authenticated, service_role; aur anon/public se revoke.

8. File / Folder Structure

```
src/
  routes/
    __root.tsx      root shell, fonts, QueryClient, AuthProvider, Toaster, 404 + error UI
    index.tsx       public landing page
    auth.tsx        login + register tabs
    dashboard.tsx    stats, upcoming deadlines, latest notices
    assignments.tsx  full CRUD, search, filters, risk badges
    notices.tsx      student notice board
    admin.tsx        admin-only: publish/delete notices, student roster
  components/
    AppShell.tsx     sidebar + header + auth guard + admin guard
    ui/*             shadcn primitives
  lib/
    auth.tsx         AuthProvider: session, profile, role, signOut
    assignments.ts   types, daysUntil, deadlineRisk, computeStats, formatters
    utils.ts         cn() helper
    integrations/supabase/ auto-generated client + types (do not edit)
    styles.css       Tailwind v4 theme, OKLCH tokens, custom utilities
    supabase/migrations/ 4 SQL migrations (schema, RLS, grants, fixes)
```

9. Auth Flow (step by step)

- User /auth par jaake Register tab me name, email, password bharta hai.

- Supabase signUp call hota hai; email confirmation auto-enabled hai isliye turant login possible.
- Database trigger profile + 'student' role create karta hai.
- AuthProvider onAuthStateChanged se session sun-ta hai aur profiles + user_roles parallel fetch karta hai.
- Role admin ho to sidebar me Admin panel link render hota hai.
- AppShell me guard: agar session nahi hai to /auth par redirect; agar requireAdmin hai aur role admin nahi to 'Access denied' card.

10. Page-by-page UI description

Landing (/)

Gradient hero with product name, one-line value proposition, primary CTA 'Get started' aur secondary 'Sign in'. Neeche 3-4 feature cards (Assignment tracking, Deadline risk, Notice board, Role-based access) aur risk-level legend. Route ka apna head() SEO title + description ke saath.

Auth (/auth)

Centered card, tabs Login / Register. Inline validation, disabled state while submitting, toast on error. Successful auth par router /dashboard bhej deta hai.

Dashboard (/dashboard)

Top par 5 compact stat cards (Total, Pending, Completed, Due this week, Overdue) with icon + number. Left column: 'Upcoming deadlines' list — subject, title, relative date ('Due today', 'In 4 days', '2 days overdue') aur risk badge. Right column: latest 3 notices preview with 'View all' link.

Assignments (/assignments)

Header action button 'Add assignment' dialog kholta hai (subject, title, description, deadline date-picker, priority select). Search box title/subject par filter karta hai, tabs se All / Pending / Completed. Har card par risk badge, priority tag, deadline, aur actions: complete toggle, edit, delete (confirm). Mutations ke baad query invalidate hoke list instantly refresh hoti hai.

Notices (/notices)

Simple chronological card list — title, message, published time. Empty state jab koi notice na ho.

Admin panel (/admin)

Do sections: (1) Publish notice form + list of published notices with delete; (2) Students table — name, email, role tag. Non-admin ko access denied screen.

11. Verification / Testing done

- Automated browser run (Playwright) se end-to-end flow test kiya gaya.
- Register → login → redirect to /dashboard verified.
- Assignment create karke HIGH risk badge automatically aana verify hua.
- Admin notice publish + student side par dikhna verify hua.
- Console errors: none. Build: successful.
- Test data (dummy assignments + notices) baad me clear kar diya gaya.

12. The Prompt Book — how this app was specified

Neeche wahi detailed prompts hain jinse yeh app banaya ja sakta hai. Inhe order me use karo.

Prompt 1 — Project setup & design system

Build a professional academic web app called "Smart Student Assignment & Deadline Manager" (StudyBoard) using React + TanStack Start, Tailwind v4 and shadcn/ui. Create a token-based design system in src/styles.css using OKLCH colours: deep navy primary, warm amber accent, green success, amber warning, blue info, red destructive, dark navy sidebar, warm off-white background. Fonts: Sora for headings, Manrope for body, loaded via a <link> in the root route head. Add surface-card and hero-gradient utilities, card + lift shadows, and a complete dark mode. Avoid generic purple-on-white AI styling.

Prompt 2 — Database & security

Set up the backend with four tables: profiles(id, name, email, created_at), user_roles(id, user_id, role) with an app_role enum ('admin','student'), assignments(id, user_id, subject, title, description, deadline date, priority, status, created_at) and notices(id, title, message, created_by, created_at). Never store the role on profiles. Add a SECURITY DEFINER has_role(_user_id, _role) function, GRANT statements for every table, enable RLS, and write policies: users manage only their own assignments, admins can read all assignments and all profiles, every signed-in user can read notices, only admins can insert/delete notices. Add a trigger on new auth users that creates the profile and assigns the default 'student' role. Grant EXECUTE on has_role to authenticated and service_role.

Prompt 3 — Deadline risk engine

Create src/lib/assignments.ts with the Assignment type and this logic: daysUntil(deadline) returns whole-day difference in local time; deadlineRisk() returns DONE if completed, OVERDUE if pending and past due, HIGH if <= 1 day left, MEDIUM if <= 3 days, otherwise LOW. Also export riskStyles (badge classes per risk), computeStats() returning total, pending, completed, dueThisWeek (0-7 days) and overdue, plus formatDate() and relativeDeadline() ("Due today", "Due tomorrow", "In 5 days", "2 days overdue").

Prompt 4 — Auth & app shell

Add an AuthProvider that tracks the session, fetches the profile and the user's roles, exposes { user, session, profile, role, loading, signOut }, and prefers 'admin' when both roles exist. Build an AppShell component with a dark fixed sidebar (logo, nav links Dashboard / Assignments / Notices / Admin panel, profile card with role and sign out), a mobile hamburger drawer with overlay, and a sticky blurred header showing page title, subtitle and an actions slot. Redirect to /auth when signed out and show an "Access denied" card when a page requires admin and the user isn't one.

Prompt 5 — Pages

```
Create these routes, each with its own SEO head():
/          landing page with gradient hero, feature cards, risk legend, CTAs
/auth      login + register tabs with clear error handling
/dashboard five stat cards, upcoming deadlines with risk badges, latest notices
/assignments full CRUD in a dialog form (subject, title, description, deadline,
           priority), live search, All/Pending/Completed tabs, complete toggle,
           edit and delete with confirmation, risk badge on every card
/notices    read-only chronological notice board
/admin      admin-only: publish and delete notices, list all students with roles
Use TanStack Query for all reads and mutations with cache invalidation,
skeleton loading states, empty states and toast feedback.
```

Prompt 6 — Polish & QA

Make everything responsive from 360px up, keep all colours as semantic tokens (no hardcoded hex or text-white in components), add accessible labels and focus rings, then run an end-to-end browser test: register, log in, create an assignment due tomorrow and confirm it shows a HIGH badge, publish an admin notice and confirm students can see it. Fix any console or permission errors and clear the test data afterwards.

13. How to run / extend

- Local run: `bun install phir bun run dev (port 8080).`
- Environment variables `.env` me hain (backend URL + publishable key) — inhe kabhi public repo me commit mat karna.
- Naya feature add karna ho to: migration → types regenerate → route file → AppShell nav link.
- Future ideas: email/push deadline reminders, file attachments per assignment, subject-wise analytics charts, calendar view, semester timetable, admin CSV export.